README File

This repository documents the partial reproduction of Figures 1–3 from Glaus et al. (2025), *Nature Genetics* (https://doi.org/10.1038/s41588-024-02026-9) [1]. This serves as a Final Project for Colorado State University's CM515 Spring 2026 course. The goals of this project were to utilize the code, data, and methods provided by the authors to reproduce Figures 1 through 3 from the publication.

Introduction

This article investigates the potential to repair a deleterious domestication variant found in tomato. Deleterious mutations have accumulated in the crop germplasm throughout the process of domestication [2]. The authors of this paper were interested in experimentally assessing the potential of precision genome editing as a method for repairing deleterious mutations in the crop germplasm. They were particularly interested in mutations in the florigen pathway, a pathway involved in the transition of shoot meristems from vegetative growth to reproductive growth [3]. Certain changes to this transition have been selected for during domestication and breeding, but research into the accumulation of deleterious mutations in the florigen pathway is lacking [1].

The florigen activation complex plays an important role in the transition to flowering by activating pertinent genes. The complex is composed of florigen proteins (members of the *CENTRORADIALIS, TERMINATING FLOWER1, SELF-PRUNING (CETS)* gene family), 14-3-3 proteins, and a bZIP transcription factor (member of the Group-A basic region/leucine zipper (bZIP) gene family) [3]. The authors predicted deleterious variants in the *CETS* and bZIP gene families, and they identified a bZIP gene with a deleterious mutation that was enriched in domesticated genomes. They studied the gene's evolutionary history and function and examined the deleterious variant's impact on the activation of target genes. They found the mutant to have reduced function, and examined potential causes of this reduced function. Finally, they asked whether base editing could be used to repair the mutation.

Overall, the authors did an excellent job of providing data, code, and thorough descriptions of their computational methods. They provided a repository, available on both Github and Zenodo, that contained many of the code scripts and data files necessary to reproduce their work

(https://github.com/soyklab/delvar-2024, https://doi.org/10.5281/zenodo.13805190). Several of their methods required a high performance computing cluster and could not be run on a local computer; however, for most such procedures, the authors provided output files so it was possible to reproduce the rest of the workflow. The authors provided a supplementary repository on Zenodo (https://doi.org/10.5281/zenodo.13787654), which contained important reference genome materials and the deleterious variant prediction data. Within the article they provided several files of supplementary/source data that were necessary for reproduction. Finally, they provided links to genomic and proteomic data available online at the Solanaceae Genomics Network (https://solgenomics.net/ftp/genomes/Solanum_pimpinellifolium/LA1589/2020/), the National Center for Biotechnology Information (NCBI) (https://www.ncbi.nlm.nih.gov/), and the Plant Transcription Factor Database (PlantTFDB) (v.5.0)[4] (https://planttfdb.gao-lab.org/).

Figure 1

Figure 1 illustrates the prediction of deleterious variants. The authors provided much of the data, code, and thorough method descriptions needed for this workflow. Annotated genome information for the reference genome (*Solanum pimpinellifolium* accession LA1589) was provided in the supplementary repository. Links to raw Nanopore and Illumina sequence data on the NCBI Sequence Read Archive (NCBI SRA) were provided. All shell scripts used to align sequences, call variants, and predict deleterious variants (sorting intolerant from tolerant (SIFT)[5] analysis) were provided in the repository. Despite all the necessary information being available for sequence alignment, variant call, and SIFT[5] analysis, I was unable to reproduce this analysis from because it is designed to be done on a high-performance computing cluster. I therefore began my reproduction at the next step, with the analysis of the variant prediction results. The authors made it quite easy to follow their workflow from this point on.

In the repository, the authors provided the necessary outputs of the variant prediction, as well as an R script used for their analysis and visualization of the data ('delvar_data-analysis.R'). The script is well commented and easy to follow. The script first loads and cleans the SIFT[5] variant annotation data, loads in identifying accession and species data, and joins these two datasets. It then generates two bar plots presenting the number of deleterious variants predicted genome-wide for each accession; one bar plot is color coded by genotype (homozygous vs heterozygous)

and one by species. These plots together represent the information seen in Figure 1A, but modification to the code was required to present the data as it is seen in Figure 1A. I wrote these modifications in a separate script, 'modifydelvarfigures.R'. While many of the modifications were simple and superficial (i.e. adjusting legend position, removing legend title), producing the color coded row of squares representing species data required a lot of brainstorming, troubleshooting, and online resources. Ultimately, I found I could accomplish this by plotting two layered bar geometries - one for genotype and one for species - and setting y = -300 for all bars in the species layer. The final figure that I produced aligns very closely with the figure produced by the authors.

This script then subsets the predicted variants by gene family (*CETS*, bZIP) and calculates how many accessions in each family carry each predicted variant. It uses this calculation to produce a dot plot of SIFT[5] scores for each gene family, with the x-axis and color both representing SIFT[5] score and dot size representing the number of accessions carrying the variant. These two plots correspond to Figures 1B and 1D. Minor modifications were required to clean up the appearance of these plots to bring them to publication quality, including adjusting the location of the legend, adding labels, setting the theme to classic, and using ggarrange()[6] to arrange the plots into one clean panel. The figures that I produced align very closely with those produced by the authors.

The script then produces barplots presenting the distribution of the predicted variants in each family across three tomato species. Each species represents a different point in the history of the domestication of tomato. Minor modifications were again required to clean up the appearance of these plots to bring them to publication quality, including adding titles, adding labels, setting the theme to classic, and using ggarrange()[6] to arrange both plots into one clean panel. The figures that I produced align very closely with those produced by the authors.

The modifications that I made in order to clean up these figures can be seen in 'KM_modifydelvarfigures.R', which is intended to be run immediately after 'delvar_data-analysis.R'. Overall, the authors did an excellent job of making it easy and accessible to reproduce Figure 1, even without access to a high performance computing cluster.

<u>Figure 2</u>

Figure 2 investigates a bZIP gene for which a putative deleterious variant was predicted in landrace and domesticated genomes. Overall, the data and code availability for this workflow is not as robust as it is for Figure 1. There are a few instances of required data or information not being available, and R scripts for data visualization were not made available.

Figure 2A is a maximum-likelihood phylogenetic tree demonstrating that the identified protein of interest is most closely related to the tomato protein SUPPRESSOR OF SP (SSP).The authors thus name this protein of interest SUPPRESSOR OF SP 2 (SSP2). Figure 2A requires the protein sequences of hundreds of bZIP proteins from tomato, *Arabidopsis*, and *Physalis*. While the authors provide a link to the PlantTFDB [4], which contains the necessary sequence data, they do not provide enough information to identify *which* bZIP proteins they use for this alignment. All tools used for this workflow (MAFFT (v.7.481) [7], IQ-Tree (v.2.2.0.5)[8], and FigTree (v.1.4.4) (http://tree.bio.ed.ac.uk/software/figtree/)) are open source (accessible) and the authors explicitly identify the parameters they used. While I could not reproduce this figure precisely without knowing which bZIP proteins to include, I did attempt an approximate reproduction. To do so, I downloaded all *Arabidopsis thaliana* and *Solanum lycopersicum* bZIP sequences available on the PlantTFDB[4]. I aligned these sequences with MAFFT[7] according to default protocols. This alignment can be seen in the fasta file 'aligned_alltomandarabbZIP.fasta' in this repository. I then attempted to construct a maximum-likelihood phylogenetic tree in IQ-Tree[8]. However, I was unable to successfully run IQ-Tree[8], despite troubleshooting efforts. Ultimately, I was unable to reproduce this figure.

For Figures 2B, 2D, and 2F, authors provided all necessary data in a well labeled and organized manner. They did not, however, provide R scripts to produce the figures. I wrote and commented the required R scripts, which can all be found in this repository. For all three plots, my results align closely with those in the publication.

Figure 2B plots the expression patterns of *SSP* and *SSP2* in different plant tissues at different developmental stages. While reproducing Figure 2B, I discovered that the authors chose to exclude some expression data, namely all leaf data (early, mid, late, transitional, and sympodial). I plotted this additional data to get a sense of what it might indicate, and it appears to further demonstrate similar expression patterns between *SSP* and *SSP2*. I am curious why the authors

chose to exclude this data, but I don't believe the exclusion alters the biological conclusions that are drawn from this analysis.

Figure 2D plots the distribution of the ancestral *SSP2* allele (*SSP2*^{S169}) and the domesticated *SSP2* allele (*SSP2*^{F169}) across several species ('Groups') that represent different points in the domestication history of tomato. The authors excluded some data from this plot as well, namely data from the Groups "Mixture", "SLC_SP_Peru", "SLC_non_Andean", and "unk". It was difficult to investigate the implications of this choice because it is not clear to what point along the domestication history of tomato all of these groups belong. I think it is likely that "Mixture", "SLC_SP_Peru", and "unknown" were excluded because they represent hybrids or unidentifiable data. However, I am unsure why the authors excluded SLC_non_Andean data, which can be assumed to be *Solanum lycopersicum* var. *cerasiforme*. This should be further investigated.

Figure 2F is a box and whisker plot illustrating the transactivation activities of SSP, SSP2^{S169}, SSP2^{F169}, and a control on genes purported to be activated during the floral transition. While my figure aligns pretty closely with the figure in the publication, there are discrepancies in the lengths of several whiskers. The authors state that the whiskers in their plot represent maximum and minimum values. However, with all data points laid over top of the box plots, it is apparent in the publication figure that the whiskers do not always reach out to the maximum values. Thus, the corresponding whiskers on my plot, which do reach out to all minimum and maximum values, are longer than those on the publication plot. My hunch is that the authors excluded outliers from their maximum and minimum value calculations but did not make this explicit in their description of the figure. Because the authors did rigorous statistical analysis of these results, I do not believe the choice to present the whiskers in this way impacts any biological conclusions. However, I do think it is important to clear up the discrepancy between the plot and its description. Additionally, the authors provide a description of their statistical analyses but they do not provide the R scripts used to run these analyses. This would be a good addition to their repository.

Figure 2C illustrates a partial alignment of several *Solanaceae* SSP and SSP2 protein sequences, along with their *Arabidopsis* homologs. While the authors provide a link to the PlantTFDB[4], which contains most of the required protein sequences, it was quite time consuming and

challenging to locate all the correct sequences. In addition, I was unable to locate the *Physalis grisea* SSP sequence despite a lot of effort. The authors access this particular sequence by performing a BLAST search on a cited *Physalis grisea* protein annotation. After a lot of digging, I successfully located this annotation file. However, I was unable to perform the BLAST search because it requires the data to be in fasta format, and I was not able to convert the file to this format. Ultimately I was unable to include this *Physalis* protein in the reproduction. The tool that the authors used to align these sequences (MAFFT (v.7.481)[7]) is open source and accessible, and the authors specified the parameters they used. I successfully performed the alignment via MAFFT [7] using the same parameters. There is a very important discrepancy between my results and the results presented in the publication. The publication demonstrates that the putative deleterious variant in the domesticated *SSP2* allele leads to a serine to phenylalanine exchange within the DNA-binding region of the protein. Importantly, the publication asserts that this exchange is seen only in *Solanum lycopersicum* (landrace/domesticated genomes). However, the alignment that I performed suggests that this S-F exchange also occurs in *Solanum pimpinellifolium*, the wild tomato genome. This is a very important discrepancy to investigate because the authors draw biological conclusions from their assessment that the deleterious variant of *SSP2* does NOT appear in wild tomato genomes. I was not able to find an error in my work that could have led to this discrepancy, so this should be further investigated.

Figure 2E provides a predicted 3D structure of the ancestral and domesticated SSP2 proteins interacting with DNA. These models illustrate that the F-S exchange likely leads to an increase in the distance between the protein and the target DNA. All data, code, and parameters required for this workflow are provided. However, I could not complete this reproduction because it requires expertise that I could not gain within the scope/timeline of this project. For the sake of reproducibility, this is an important figure to reproduce because the authors proceed from this model to hypothesize that the proposed increase in distance between protein and DNA leads to a reduction in the function of SSP2 as a transcription factor.

Figure 3

Figure 3 illustrates the reduced ability of the domesticated SSP2 protein to bind to target DNA. DNA affinity purification sequencing (DAP–seq) was conducted to examine and compare the

DNA binding activity of the three transcription factors (TFs) SSP, $SSP2^{S169}$, and $SSP2^{F169}$. >14,000 peaks were identified that were significantly enriched, and peak sharing by the TFs was assessed. Additionally, for each TF, genes with proximal peaks (identified as ≤3 kilobases (kbp) upstream and ≤2 kbp downstream) were identified and compared. Figures 3A and 3D illustrate these comparisons in the form of Venn diagrams. The data and R code required to produce these figures were both provided and made easily accessible. 'dapseq-analysis.R' is designed to be run after two other scripts ('csaw_diff_binding_analysis.R', 'compare_peaks.R') that analyze the DAP-seq data (using an HPC) and provide outputs that are usable on a local computer. The authors provide one such output (peak data, 'peak_table_fdr.0.01.tsv'), and by doing so they make it possible to run 'dapseq-analysis.R' without access to an HPC. The script imports peak data, cleans the data, and filters the data for proteins of interest. It then visualizes, in Venn diagrams, the gene and peak overlaps between the three transcription factors. The script ('dapseq-analysis.R') required modifications to make the figures presentable (see 'CleanFigs3A3D.R').

Figure 3B was more difficult to produce. I had trouble determining what data was required to produce this figure, so I took this opportunity to practice working with AI (Claude). Claude explained that I needed (1) the peak data provided in 'peak_table_fdr.0.01.tsv' and (2) gene model data for the SollycSweet genome, which is available from the Solanaceae Genomics Network. I asked Claude to write a script to produce Figure 3B. The resulting script that Claude provided was quite impressive, but it required a lot of modifying and troubleshooting to produce the desired final product. Below is a noncomprehensive list of some changes I had to make:

- Adjusted a few functions that were using out-of-date syntax.
- Converted features to factors and releveled factors so they appear in desired order on plot.
- Renamed features using gsub() so that features group correctly and appear in the legend with clean labels.
- Made some changes to the plot layout to clean it up and align it more closely with the publication figure.
- In the first figure produced, the data was clearly incorrect, so I asked Claude to help me figure out the problem; after going back and forth we figured out that the script was

sorting the promoters into incorrect bins; we modified the annotatePeak() calls and changed the subsetting of the data in order to fix the problem.

- I did not like the colors automatically assigned to the features, so I asked Claude how I could assign the colors manually; Claude changed the script so that it built the plot manually in ggplot2[10], and this allowed me to assign colors within ggplot2[10].

The final script after modifications ('Claude_KM_Figure_3b.R') (1) loads the DAP-seq peak data from 'peak_table_fdr_0.01.tsv' and filters the data using the same parameters used by the authors, (2) builds a gene model database from the SollycSweet100 genome annotation and uses it to classify each peak by genomic feature (via ChIPseeker[11]), (3) cleans the data and organizes the feature categories to match those used in the paper, and (4) plots the data as a stacked bar chart using ggplot()[10], with each bar representing a different protein and each section of a bar representing the proportion of peaks in a given genomic feature category. With this final script I was able to produce a figure that aligns very closely with the publication figure.

Figure 3C illustrates de novo motif enrichment analysis of the DAP–seq peak regions for the three TFs. The BED files required to run this analysis are not all available: the files required to assess $SSP2^{S169}$ and $SSP2^{F169}$ were previously produced by 'dapseq-analysis.R', but the same files required to assess SSP were not produced. I was thus only able to analyze motifs for the SSP2 variants. I employed Claude to write a script to produce this figure. The script required modifications, but after working with Claude to problem solve I was able to produce the motif visualizations. The code used for this reproduction can be found in 'Claude_fig3C_motif_analysis.sh'; it employs bedtools [12] and Multiple Em for Motif Elicitation (MEME, an interactive tool for viewing results)[13]. For each TF, the analysis produced a MEME HTML file with 3 motifs. For $SSP2^{S169}$, one motif produced by my analysis aligns fairly closely with the publication figure, though there are some discrepancies. For $SSP2^{F169}$, none of the motifs that I produced align closely with the publication figure. Due to time constraints, I was not able to thoroughly investigate potential causes of these discrepancies, but this should definitely be pursued further. Their results suggest there is only a subtle variation for $SSP2^{F169}$ (and that this variation is outside the core motif), but my results suggest no motif is shared across the two variants.

Figures 3E through 3H require the extraction of data from the BigWig files provided in the supplementary repository. I employed Claude again to help me understand what I needed to do to produce these figures. Claude first suggested using the rtracklayer[14] package in R, which is used for importing and managing genomic annotations. However, the package's UCSC kent library was incompatible with Windows. Claude recommended some workarounds, all of which I tried and failed to employ despite extensive troubleshooting. Claude then recommended a deepTools-based [15] pipeline in WSL. I attempted this but continually ran into problems that I could not solve despite troubleshooting. I was ultimately unable to produce these figures. Figure 3E illustrates genome-wide DAP-seq coverage for all three TFs. A partial reconstruction of 3E is possible with the BED files provided (authors provide some but not all the files required), but I did not pursue this due to time constraints after losing time troubleshooting the above approaches.

Gene Ontology Analysis

The authors performed a gene ontology enrichment analysis to detect enriched gene ontology terms for genes near the private peaks of each TF (SSP, $SSP2^{F169}$, and $SSP2^{S169}$). This analysis is performed in 'dapseq-analysis.R' (which employs clusterProfiler[16] to analyze gene ontology data), but it requires the prior production of a custom gene ontology database, which is done in 'AnnotationForge.R'[17].

When I began this project, I did not gain a super clear picture of the workflow before diving right into the code and data. Instead, I got a big picture grasp of the workflow and thought I could pick up the details as I moved through it. This was unfortunately quite an inefficient method. It led me to spend an inordinate amount of time on this gene ontology analysis, only to later discover that it was not necessary for the scope of this project; the gene ontology analysis is not presented anywhere in Figures 1 through 3. For the sake of testing reproducibility, it is absolutely a great idea to run this analysis. However, for the sake of setting priorities, this ultimately created a problem for me down the line by limiting the time I had available for deliverables farther down the pipeline.

The Annotation Forge[17] script imports data from a file that the authors do not provide, and I spent an enormous amount of time looking for this file. After far too long, I finally employed the

help of AI (ChatGPT). ChatGPT reported that this file in fact did NOT exist; rather, it was likely a file the authors had created that compiled information from multiple files on the ITAG4.0 database[9]. Upon learning this, I looked closely at exactly what data was being used from this file, and I realized I could import this information from a different ITAG4.0[9] file ('ITAG4.0_gene_models.gff'). I reworked the script to interact with this file instead and was able to produce the same output. These modifications can be seen in 'KM_AnnotationForgeModified.R'. In the case of this gene ontology analysis, the authors made it very difficult to follow their script.

Conclusion

Overall, the authors did a great job of making their work reproducible. Most data and code was available, but there were a few major hang ups that created difficulties; on my end, I underestimated the time that such hang ups could take. While most code and data was available, I think the authors could improve the organization and labeling of their data, code, workflows, etc. The information is spread across two online repositories, the Data Availability section, the Supplementary Information section, and the Source Data section. I am not yet familiar with established conventions for supplying this information, but I feel the material would be much more accessible if it was all available in one organized place (i.e. the Github repository). Furthermore, the scripts could be commented better to provide clear information about the purpose, required inputs, and outputs of each script; the authors do provide this information for some scripts but not all. Improving their organization and labeling in this way would assist greatly with clarity and would reduce the time required to reproduce this work.

Through the process of testing the reproducibility of this work, I have learned a lot about how to ensure my own work is reproducible. I will definitely provide all raw data and links to any data pulled from repositories, just as these authors did. However, I will label, organize, and annotate my work more thoroughly to ensure that it is easy to understand and sort through.

This project provided a great learning experience, and I have some major takeaways, almost all related to efficiency:

- Become thoroughly familiar with the workflow and set priorities prior to diving into the project.
- There are an endless amount of rabbit holes to get sucked down that can easily consume more time than they are worth.
- I should get on board with AI: I refused to use AI until toward the end of this project, but utilizing AI sooner could have saved me quite a bit of time in troubleshooting, problem solving, etc. I have realized that becoming literate in how to use AI assistance with coding projects will be important to my success as a biologist.
- In terms of testing reproducibility, I focused far too much time on making the figures 'pretty' and very similar to the figures in the publication. I should have focused more on critically examining the workflows, code, and data provided by the authors; by the time I had finished polishing the figures, however, I had limited time to do so. While this provided me with a lot of R practice (a goal of mine) it led me astray from the primary goal of assessing reproducibility.
- Gained familiarity with genomics data processing, file types, etc.
- Comment code while writing the code rather than after the fact - it is much more efficient.

Future steps for this project would include addressing all discrepancies between my analyses and the author's analyses, investigating why the authors left data out of some figures, and using an HPC to reproduce the parts of the pipeline I was unable to perform on my personal computer.

AI Usage Statement

I used AI (Claude, ChatGPT, and Google Gemini) to assist with troubleshooting and problem solving and to answer questions I had about syntax, packages, etc. I also used Claude to write two scripts used in this project, 'Claude_KM_Figure_3b.R' and 'Claude_fig3C_motif_analysis.sh'; I made modifications to these to produce working scripts.

References

[1] Glaus, A.N., Brechet, M., Swinnen, G. *et al*. Repairing a deleterious domestication variant in a floral regulator gene of tomato by base editing. *Nat Genet* **57**, 231–241 (2025). https://doi.org/10.1038/s41588-024-02026-9

[2] Sebastien Renaut, Loren H. Rieseberg, The Accumulation of Deleterious Mutations as a Consequence of Domestication and Improvement in Sunflowers and Other Compositae Crops, *Molecular Biology and Evolution*, Volume 32, Issue 9, September 2015, Pages 2273–2283, https://doi.org/10.1093/molbev/msv106

[3]Gao, H., Ding, N., Wu, Y. *et al.* Florigen activation complex forms via multifaceted assembly in *Arabidopsis*. *Nature* **648**, 686–695 (2025). https://doi.org/10.1038/s41586-025-09704-6

[4]Jinpu Jin, Feng Tian, De-Chang Yang, Yu-Qi Meng, Lei Kong, Jingchu Luo, Ge Gao, PlantTFDB 4.0: toward a central hub for transcription factors and regulatory interactions in plants, *Nucleic Acids Research*, Volume 45, Issue D1, January 2017, Pages D1040–D1045, https://doi.org/10.1093/nar/gkw982

[5]Vaser, R., Adusumalli, S., Leng, S. *et al.* SIFT missense predictions for genomes. *Nat Protoc* **11**, 1–9 (2016). https://doi.org/10.1038/nprot.2015.123

[6]Kassambara A (2026). _ggpubr: 'ggplot2' Based Publication Ready Plots_. doi:10.32614/CRAN.package.ggpubr

<https://doi.org/10.32614/CRAN.package.ggpubr>, R package version 0.6.3,

<https://CRAN.R-project.org/package=ggpubr>.

[7]Kazutaka Katoh, Daron M. Standley, MAFFT Multiple Sequence Alignment Software Version 7: Improvements in Performance and Usability, *Molecular Biology and Evolution*, Volume 30, Issue 4, April 2013, Pages 772–780, https://doi.org/10.1093/molbev/mst010

[8]Bui Quang Minh, Heiko A Schmidt, Olga Chernomor, Dominik Schrempf, Michael D Woodhams, Arndt von Haeseler, Robert Lanfear, IQ-TREE 2: New Models and Efficient Methods for Phylogenetic Inference in the Genomic Era, *Molecular Biology and Evolution*, Volume 37, Issue 5, May 2020, Pages 1530–1534, https://doi.org/10.1093/molbev/msaa015

[9]Michiel Van Bel, Francesca Silvestri, Eric M Weitz, Lukasz Kreft, Alexander Botzki, Frederik Coppens, Klaas Vandepoele, PLAZA 5.0: extending the scope and power of comparative and functional genomics in plants, *Nucleic Acids Research*, Volume 50, Issue D1, 7 January 2022, Pages D1468–D1474, https://doi.org/10.1093/nar/gkab1024

[10]H. Wickham. ggplot2: Elegant Graphics for Data Analysis. Springer-Verlag New York, 2016.

[11]Qianwen Wang, Ming Li, Tianzhi Wu, Li Zhan, Lin Li, Meijun Chen, Wenqin Xie, Zijing Xie, Erqiang Hu, Shuangbin Xu, Guangchuang Yu. Exploring epigenomic datasets by ChIPseeker. Current Protocols. 2022, 2(10): e585.

[12]Aaron R. Quinlan, Ira M. Hall, BEDTools: a flexible suite of utilities for comparing genomic features, *Bioinformatics*, Volume 26, Issue 6, March 2010, Pages 841–842, https://doi.org/10.1093/bioinformatics/btq033

[13]Timothy L. Bailey, James Johnson, Charles E. Grant, William S. Noble, The MEME Suite, *Nucleic Acids Research*, Volume 43, Issue W1, 1 July 2015, Pages W39–W49, https://doi.org/10.1093/nar/gkv416

[14]M. Lawrence, R. Gentleman, V. Carey: "rtracklayer: an {R} package for interfacing with genome browsers".

Bioinformatics 25:1841-1842.

[15]Fidel Ramírez, Friederike Dündar, Sarah Diehl, Björn A. Grüning, Thomas Manke, deepTools: a flexible platform for exploring deep-sequencing data, *Nucleic Acids Research*, Volume 42, Issue W1, 1 July 2014, Pages W187–W191, https://doi.org/10.1093/nar/gku365

[16]T Wu, E Hu, S Xu, M Chen, P Guo, Z Dai, T Feng, L Zhou, W Tang, L Zhan, X Fu, S Liu, X Bo, and G Yu.

clusterProfiler 4.0: A universal enrichment tool for interpreting omics data. The Innovation. 2021, 2(3):100141

[17]Carlson M, Pagès H (2025). _AnnotationForge: Tools for building SQLite-based annotation data packages_. R

package version 1.52.0, <https://bioconductor.org/packages/AnnotationForge>.

<u>Additional References (not cited in text but used in code or presentation materials)</u>

[18]package ‘tidyverse’

Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R, Grolemund G, Hayes A, Henry L, Hester J,

Kuhn M, Pedersen TL, Miller E, Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi K,

Vaughan D, Wilke C, Woo K, Yutani H (2019). “Welcome to the tidyverse.” _Journal of Open Source

Software_, *4*(43), 1686. doi:10.21105/joss.01686 <https://doi.org/10.21105/joss.01686>.

[19]package ‘ggthemes’

Arnold J (2025). _ggthemes: Extra Themes, Scales and Geoms for 'ggplot2'_.

doi:10.32614/CRAN.package.ggthemes <https://doi.org/10.32614/CRAN.package.ggthemes>, R package version

5.2.0, <https://CRAN.R-project.org/package=ggthemes>.

[20]package ‘reshape2’

Wickham H (2007). “Reshaping Data with the reshape Package.” _Journal of Statistical Software_, *21*(12),

1-20. <https://www.jstatsoft.org/v21/i12/>.

[21]package ‘stringr’

Wickham H (2025). _stringr: Simple, Consistent Wrappers for Common String Operations_.

doi:10.32614/CRAN.package.stringr <https://doi.org/10.32614/CRAN.package.stringr>, R package version

1.6.0, <https://CRAN.R-project.org/package=stringr>.

[22]package ‘dplyr’

Wickham H, François R, Henry L, Müller K, Vaughan D (2026). _dplyr: A Grammar of Data Manipulation_.

doi:10.32614/CRAN.package.dplyr <https://doi.org/10.32614/CRAN.package.dplyr>, R package version 1.2.1,

<https://CRAN.R-project.org/package=dplyr>.

[23]package ‘plyr’

Hadley Wickham (2011). The Split-Apply-Combine Strategy for Data Analysis. Journal of Statistical

Software, 40(1), 1-29. URL https://www.jstatsoft.org/v40/i01/.

[24]viridis/viridisLite

Simon Garnier, Noam Ross, Robert Rudis, Antônio P. Camargo, Marco Sciaini, and Cédric Scherer (2024).

viridis(Lite) - Colorblind-Friendly Color Maps for R. viridis package version 0.6.5.

[25]ChIPseeker

Qianwen Wang, Ming Li, Tianzhi Wu, Li Zhan, Lin Li, Meijun Chen, Wenqin Xie, Zijing Xie, Erqiang Hu,

Shuangbin Xu, Guangchuang Yu. Exploring epigenomic datasets by ChIPseeker. Current Protocols 2022, 2(10):

e585

Guangchuang Yu, LiGen Wang, and QingYu He. ChIPseeker: an R/Bioconductor package for ChIP peak

annotation, comparison and visualization. Bioinformatics 2015, 31(14):23822383

[26]package ‘GenomicFeatures’

Lawrence M, Huber W, Pag\`es H, Aboyoun P, Carlson M, et al. (2013) Software for Computing and Annotating

Genomic Ranges. PLoS Comput Biol 9(8): e1003118. doi:10.1371/journal.pcbi.1003118

[27]package ‘GenomicRanges’

Lawrence M, Huber W, Pag\`es H, Aboyoun P, Carlson M, et al. (2013) Software for Computing and Annotating

Genomic Ranges. PLoS Comput Biol 9(8): e1003118. doi:10.1371/journal.pcbi.1003118

[28]package ‘AnnotationDbi’

Pagès H, Carlson M, Falcon S, Li N (2025). _AnnotationDbi: Manipulation of SQLite-based annotations in

Bioconductor_. R package version 1.72.0, <https://bioconductor.org/packages/AnnotationDbi>.

[29]package ‘readxl’

Wickham H, Bryan J (2025). _readxl: Read Excel Files_. doi:10.32614/CRAN.package.readxl

<https://doi.org/10.32614/CRAN.package.readxl>, R package version 1.4.5,

<https://CRAN.R-project.org/package=readxl>.

[30]package ‘ggh4x’

van den Brand T (2025). _ggh4x: Hacks for 'ggplot2'_. doi:10.32614/CRAN.package.ggh4x

<https://doi.org/10.32614/CRAN.package.ggh4x>, R package version 0.3.1,

<https://CRAN.R-project.org/package=ggh4x>.

[31]package ‘S4Vectors’

Pagès H, Lawrence M, Aboyoun P (2026). _S4Vectors: Foundation of vector-like and list-like containers in

Bioconductor_. R package version 0.48.1, <https://bioconductor.org/packages/S4Vectors>.

[32]package ‘tidyr’

Wickham H, Vaughan D, Girlich M (2025). _tidyr: Tidy Messy Data_. doi:10.32614/CRAN.package.tidyr

<https://doi.org/10.32614/CRAN.package.tidyr>, R package version 1.3.2,

<https://CRAN.R-project.org/package=tidyr>.

[33]package ‘egg’

Auguie B (2019). _egg: Extensions for 'ggplot2': Custom Geom, Custom Themes, Plot Alignment, Labelled

Panels, Symmetric Scales, and Fixed Panel Size_. doi:10.32614/CRAN.package.egg

<https://doi.org/10.32614/CRAN.package.egg>, R package version 0.4.5,

<https://CRAN.R-project.org/package=egg>.

[34]‘grid’ package

R Core Team (2025). _R: A Language and Environment for Statistical Computing_. R Foundation for

Statistical Computing, Vienna, Austria. <https://www.R-project.org/>.

[35]eulerr

Larsson J, Gustafsson P (2018). “A Case Study in Fitting Area-Proportional Euler Diagrams with Ellipses

Using eulerr.” In _Proceedings of International Workshop on Set Visualization and Reasoning_, volume

2116, 84-91. <https://ceur-ws.org/Vol-2116/paper7.pdf>.

[36]package ‘VennDiagram’

Chen, H., Boutros, P.C. VennDiagram: a package for the generation of highly-customizable Venn and Euler

diagrams in R. BMC Bioinformatics 12, 35 (2011). https://doi.org/10.1186/1471-2105-12-35

[37]package ‘pheatmap’

Kolde R (2025). _pheatmap: Pretty Heatmaps_. doi:10.32614/CRAN.package.pheatmap

<https://doi.org/10.32614/CRAN.package.pheatmap>, R package version 1.0.13,

<https://CRAN.R-project.org/package=pheatmap>.

[38]package ‘enrichplot’

Yu G (2026). _enrichplot: Visualization of Functional Enrichment Result_. R package version 1.30.5,

<https://yulab-smu.top/contribution-knowledge-mining/>.

[39] package ‘RColorBrewer’

Neuwirth E (2022). _RColorBrewer: ColorBrewer Palettes_. doi:10.32614/CRAN.package.RColorBrewer

<https://doi.org/10.32614/CRAN.package.RColorBrewer>, R package version 1.1-3,

<https://CRAN.R-project.org/package=RColorBrewer>.

[40]package ‘gridExtra’

Auguie B (2017). _gridExtra: Miscellaneous Functions for "Grid" Graphics_.

doi:10.32614/CRAN.package.gridExtra <https://doi.org/10.32614/CRAN.package.gridExtra>, R package version

2.3, <https://CRAN.R-project.org/package=gridExtra>.

[41]package ‘EnhancedVolcano’

Blighe K, Rana S, Lewis M (2025). _EnhancedVolcano: Publication-ready volcano plots with enhanced

colouring and labeling_. R package version 1.28.2, <https://github.com/kevinblighe/EnhancedVolcano>.

[42]package ‘multcompView’

Graves S, Piepho H, Selzer L (2026). _multcompView: Visualizations of Paired Comparisons_.

doi:10.32614/CRAN.package.multcompView <https://doi.org/10.32614/CRAN.package.multcompView>, R package

version 0.1-11, <https://CRAN.R-project.org/package=multcompView>.

[43]Li, X., Liu, H. How the florigen activation complex orchestrates flowering. *Nat. Plants* **12**, 269–270 (2026). https://doi.org/10.1038/s41477-026-02226-7